Technological Institute of the Philippines – Quezon City

College of Engineering and Architecture

Computer Engineering

Tooth Fairy: A Dental Clinic Appointment Manager System

Course: CPE 010
CPE21S4 | Group 3

Andoy, Francis Nikko I.
Pulgado, First Name
Punay, Heidee S.
Santiago, David Owen A.

TABLE OF CONTENTS

THE PROBLEM

Inefficiencies in the conventional appointment management systems that many clinics utilize have been made evident by the rising demand for dental treatments. The majority of dental clinics still use manual scheduling techniques, like phone calls or walk-in consultations, which require careful paper recording and manual coordination (Azzali et al., 2024). This method frequently results in duplicate data entry, security issues, and scheduling mistakes, which adds time to the process for both staff and patients. Additionally, managing patient files by hand raises the possibility of lost or duplicate records, risking the integrity of the data. For patients who prefer digital interaction while scheduling appointments, the lack of automation further restricts accessibility and convenience. As a result, the traditional method is out of step with the increasing demands for modern healthcare services to be efficient and convenient.

Patient satisfaction and the general productivity of dental clinics are both impacted by the inefficiencies of manual appointment booking methods. According to Gomez et al. (2023), accurate appointment capacity calculations and staffing allocation are critical to sustaining hospital management quality and are difficult to do without digital technology. Similar to this, Ho et al. (2024) noted that a large number of dental clinics continue to use paper-based records, which restricts cooperation between dental specialists and makes it difficult to obtain patient information quickly. Additionally, clinics frequently struggle to improve appointment processes to ensure accessibility and efficiency (Alahmad and Alnafea, 2023, as referenced in Sihombing, 2024). Long wait times, scheduling disputes, and general discontent among patients and dental professionals are all caused by these operational difficulties. Therefore, an automated system that can resolve these problems and improve workflow efficiency is obviously needed.

To address these persistent problems, the proposed Dental Clinic Appointment Manager System was created with the goal of automating dental appointment scheduling and administration through the use of suitable data structures and algorithms in order to address these enduring issues. According to Ariyanto et al. (2022, as quoted in Sihombing, 2024), the system can effectively store, retrieve, and manage patient data while guaranteeing accuracy and security by utilizing information technology improvements. By lowering manual labor and limiting human mistakes, it will expedite appointment scheduling. Additionally, by employing algorithms to efficiently manage staff schedules and time slots, the system will optimize resource allocation. Better overall clinic management, shorter wait times, and an enhanced patient experience will

follow from this. In the end, the suggested system is a cutting-edge approach that combines healthcare administration and technology to provide more dependable and effective dental care.

OBJECTIVES

The general objective of this project is to make a dental clinic appointment manager that improves the efficiency and convenience for the patient in appointment scheduling and patient record management in dental clinics. This system aims to ease clinic operations, reduce manual errors, and improve the overall experience for both staff and patients by using data structure algorithms.

Specifically, the project aims to:

1. To develop a system for storing and managing patient information
2. Provide a scheduling system that allows doctors and staff to manage appointments.
3. Ensure proper documentation and tracking of patient treatments and medical histories
4. Enable staff to quickly and accurately search for patient records and appointment details.
5. To develop an easy-to-use system interface that enables ease of navigation of patient records and appointment schedules.

RATIONALE

The algorithmic and data structure choices for the dental appointment management system were decided with a focus on simplicity, clarity, and maintainability, specifically targeting the operational scale of a small clinic. This approach prioritizes robust, standard-library components over complex, highly-specialized optimizations that would introduce unnecessary overhead for the intended dataset size.

For primary data storage, std::vector was selected for managing the patientList and appointmentQueue. This was chosen due to its flexibility for dynamic data, simplicity of use, and sufficient performance characteristics for the anticipated scale of operations. Textual data is uniformly handled by std::string. For associative lookups, std::unordered_map provides an efficient mechanism. It is employed to track used IDs in the GenerateUnique4DigitId function, offering average-case O(1) lookup, and to map doctor identifiers to their fixed work schedules in doctorSchedules. This map-based approach allows for quick retrieval of a doctor's availability, which is stored as a std::vector of std::pair<int, int> ranges representing minutes-of-day.

Algorithmically, the system relies on focused routines that utilize standard C++ library features. Time normalization is encapsulated in the TimeStringToMinutes function, which parses varied user time strings into a consistent integer representation (minute-of-day), insulating the rest of the system from formatting inconsistencies. For data integrity, unique ID generation employs a hybrid strategy: a fast std::mt19937 random generator for the average case, with a linear-scan fallback to guarantee uniqueness in all scenarios. Core list operations for searching and deletion utilize standard library algorithms, std::find_if and the erase-remove idiom, respectively, to ensure correctness and maintainability. Similarly, appointment ordering is implemented using std::sort with a lambda comparator based on the TimeStringToMinutes output.

The central appointment availability logic iterates through a doctor's schedule and performs a linear scan of the appointmentQueue to check for exact-minute conflicts. A linear scan is chosen due to the smaller scale of the operation, which means more complex searching algorithms would be unnecessary. While the theoretical worst-case complexity of this check is O(Doctors * Appointments), this direct implementation was deemed an acceptable trade-off for its clarity and simplicity. The overall performance

profile, characterized by linear scans (O(n)) and log-linear sorts (O(n log n)), is fully aligned with the system's design goal: to provide a clear, robust, and maintainable solution for a small clinic dataset.

ALGORITHM FLOW

This section contains the algorithm's flow with corresponding flowcharts for each section of the program. It begins with a main menu that prompts the user to choose between four options: add new patient, patient list, patient appointment schedule, and exit. The flow is as follows:

Main Algorithm Flow

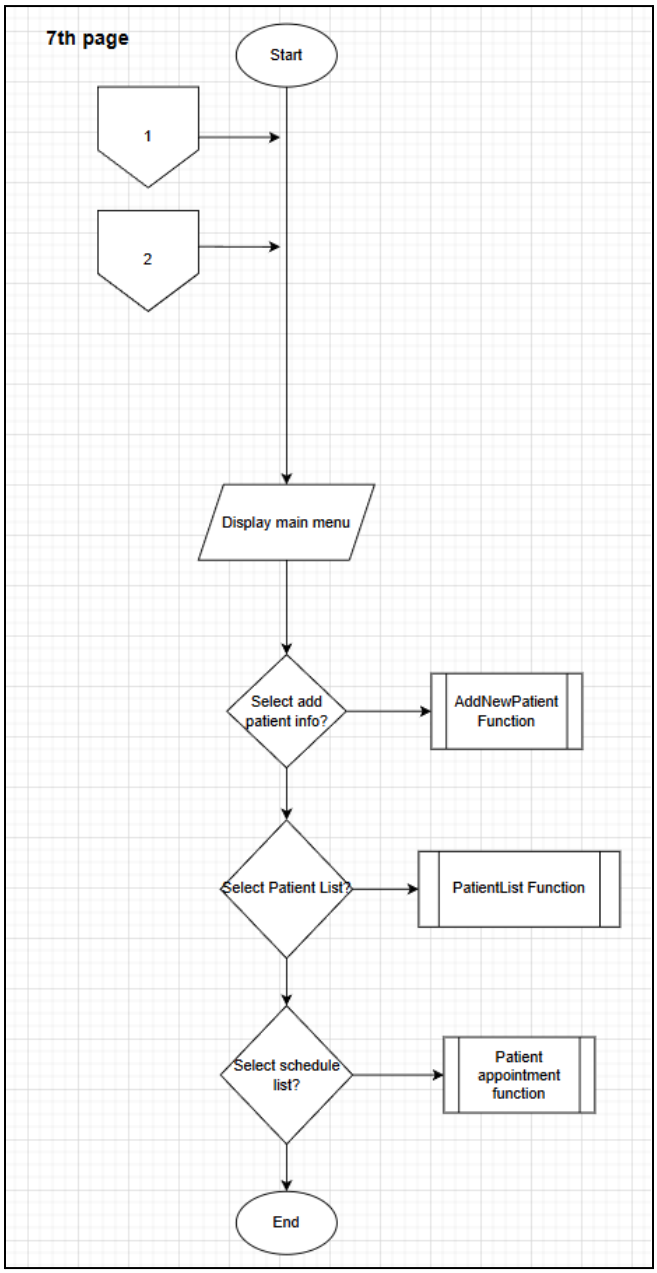

Figure 1 – Overall flow of the program

Option 1: AddNewPatient Function

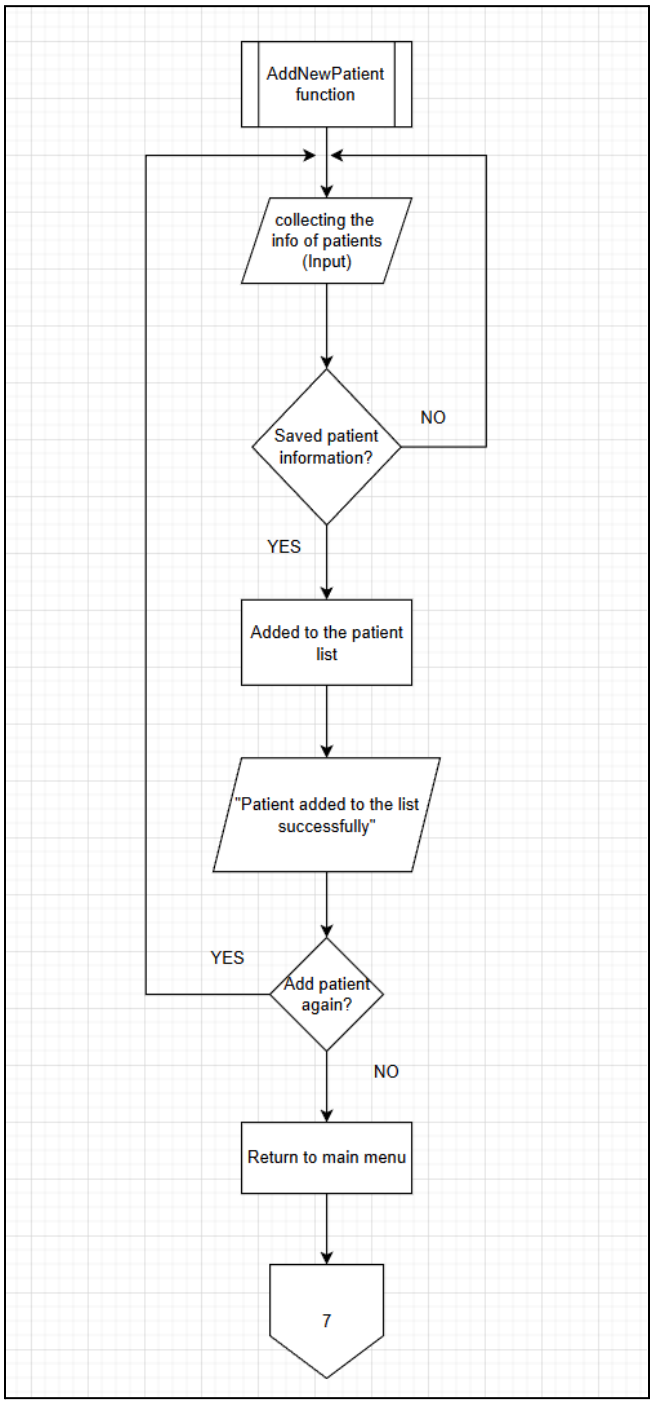

Figure 2 — Choosing add new patient

Option 2: Patient List Function

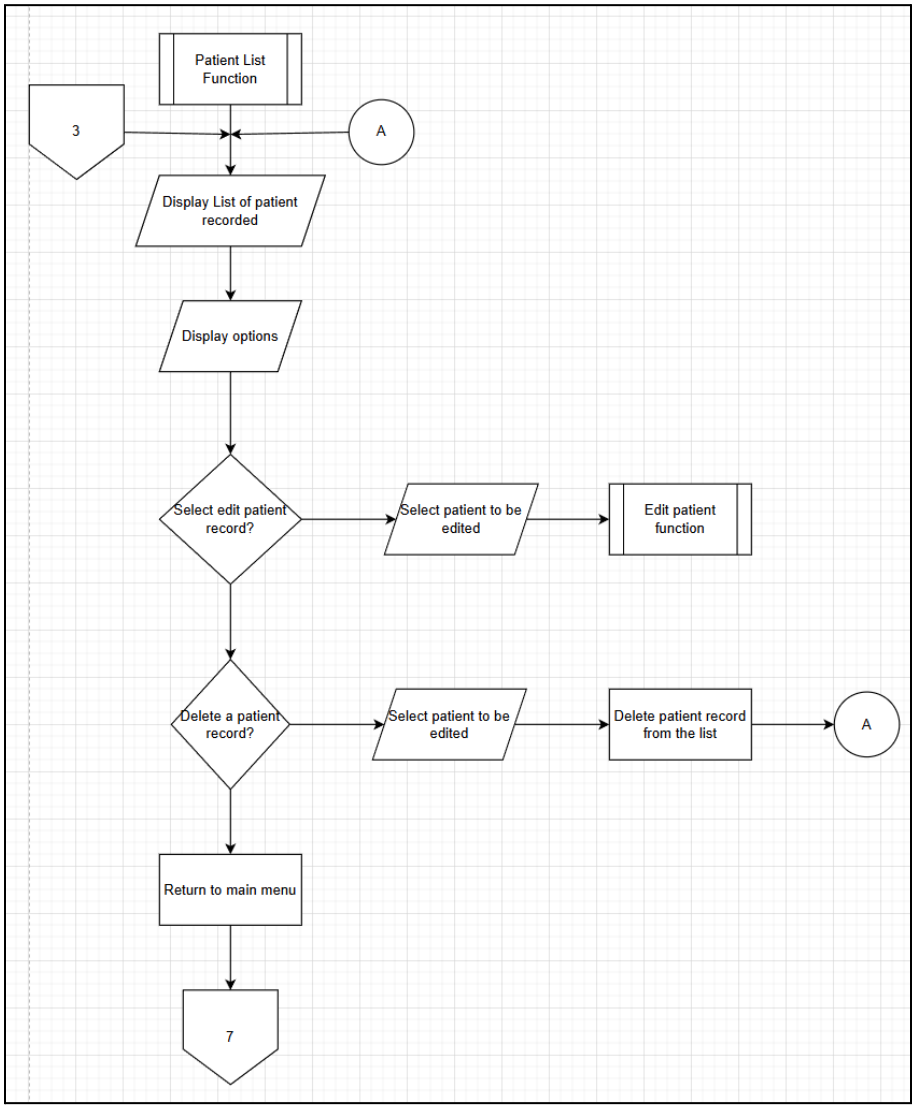

Figure 3 – Viewing patient list

In Option 3: Edit Patient Record Function

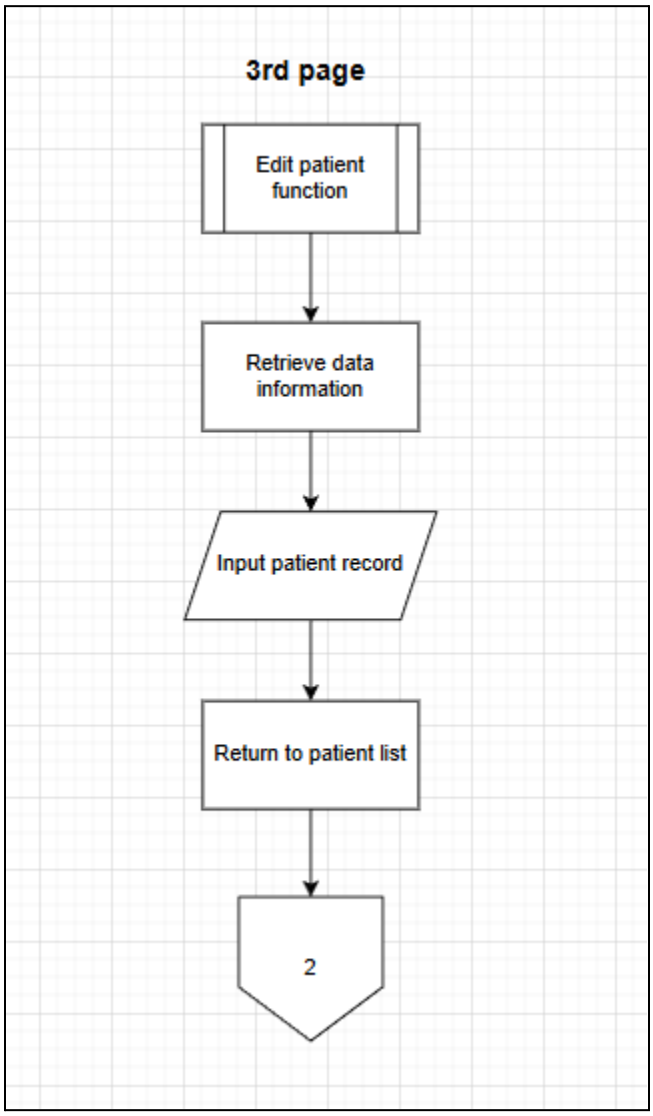

Figure 4 — Editing patient record function

Option 4: Patient Appointment Function

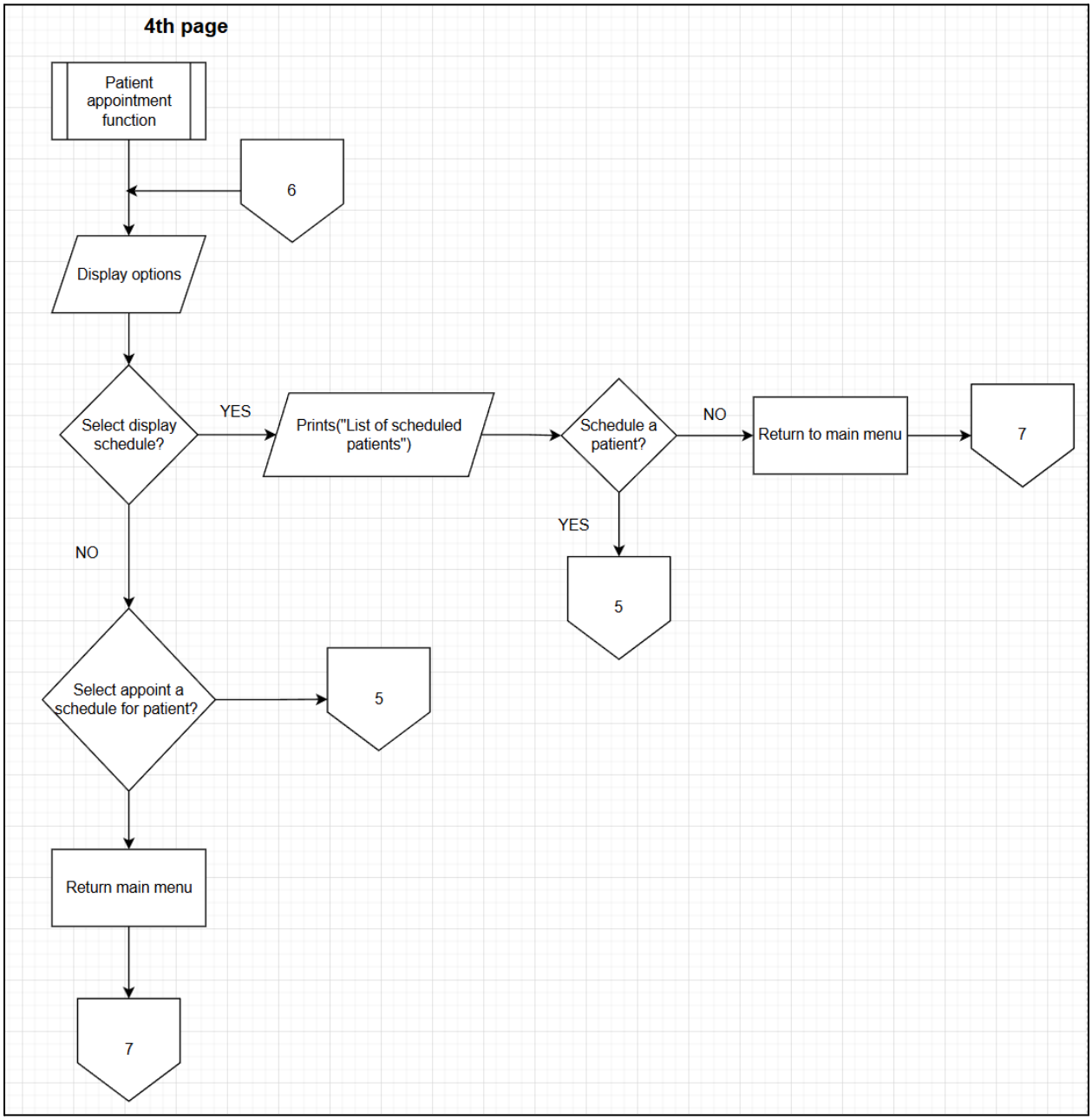

Figure 5 — Choosing an appointment schedule for the current patient.

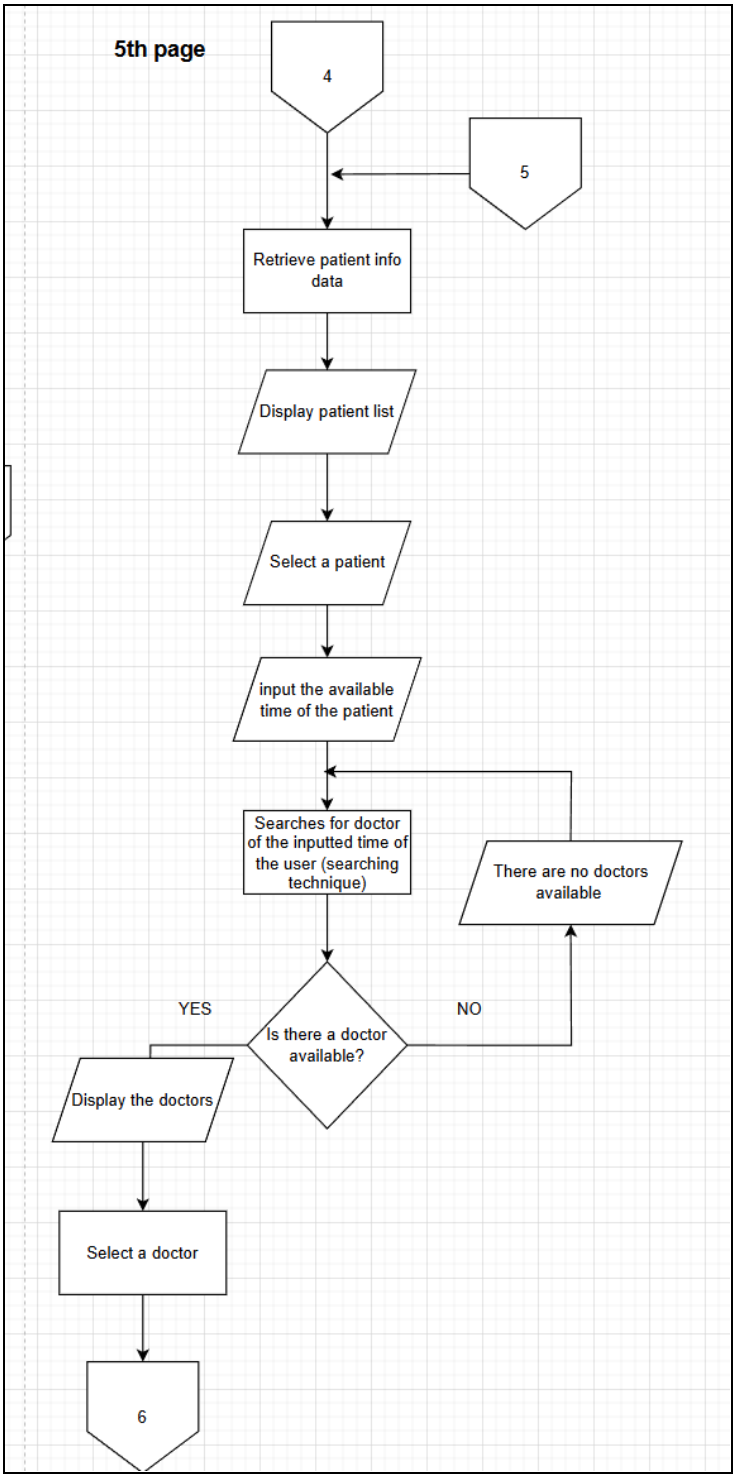

Figure 6 — Selecting patient and schedule to be appointed

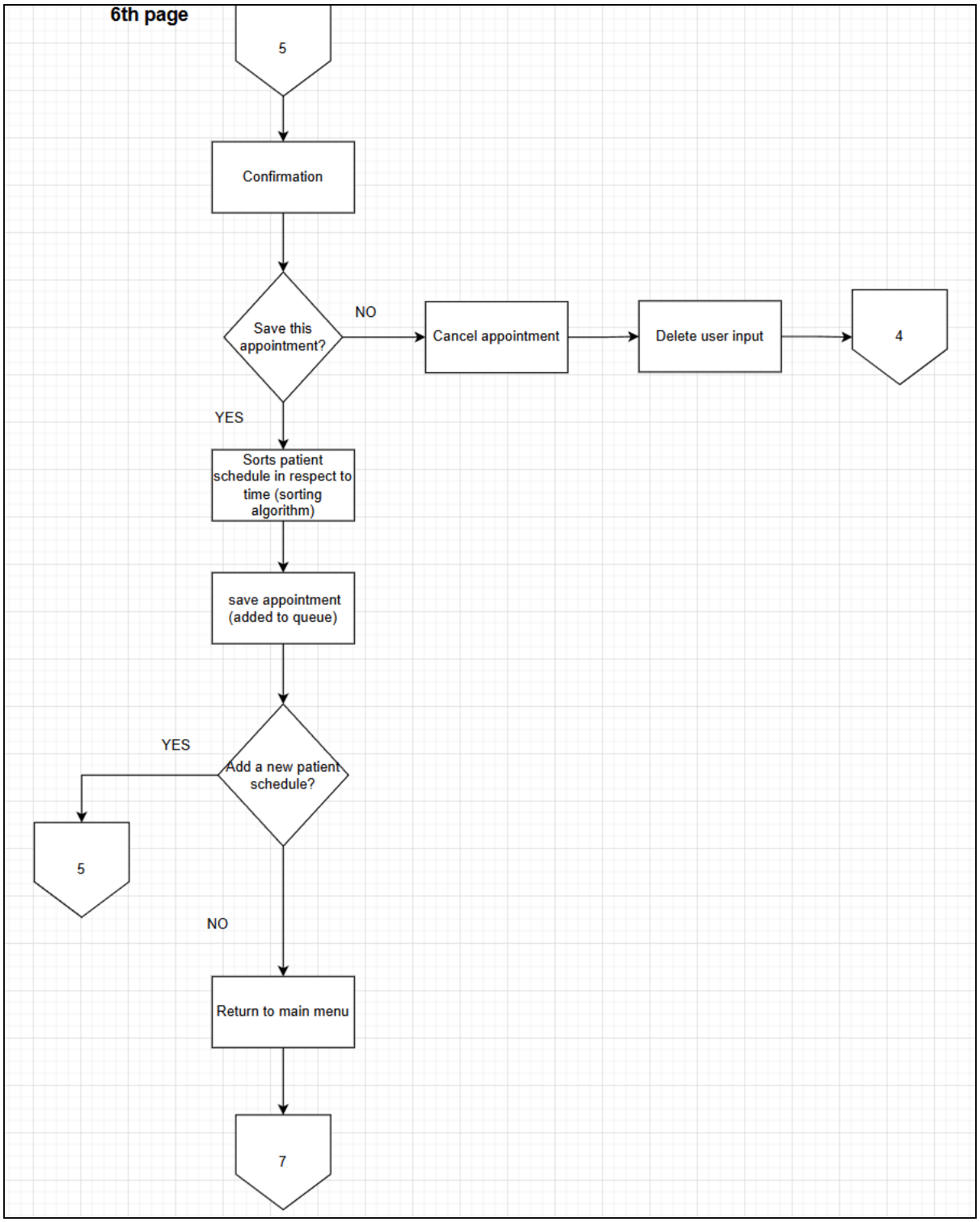

Figure 7 — Confirmation of patient schedule and return to main project flow

CODES AND OUTPUT

The following are the three code files used in the project. It consists of the main driver function in figure 8, the patient header file consisting of the patient and appointment classes in figure 9, and the patient system header file in figure 10 that consists of the main functions and the system simulation.

```cpp
#include <iostream>
#include "PatientSystem.h"

int main() {
    PatientSystem system;
    system.DisplayMainMenu();
    return 0;
}
```

Figure 8: main.cpp

```cpp
#ifndef PATIENT_H
#define PATIENT_H

#include <string>

class Patient {
public:
    int id;
    std::string name;
    int age;
    std::string contactNumber;
    std::string gender;
    std::string history;
};

class Appointment {
public:
```

```
    int patientId;
    std::string time;
    std::string doctor;
};

#endif
```

Figure 9: Patient.h

```
#ifndef PATIENTSYSTEM_H
#define PATIENTSYSTEM_H

#include "Patient.h"
#include <vector>
#include <iostream>
#include <limits>
#include <algorithm>
#include <random>
#include <unordered_map>
#include <sstream>
#include <cctype>

class PatientSystem {
private:
    std::vector<Patient> patientList;
    std::vector<Appointment> appointmentQueue;

    void EditPatientRecord(int idToEdit);
    void ScheduleAppointment();
    void AddNewPatientFunction();
    void PatientListFunction();
    void PatientAppointmentFunction();

public:
    PatientSystem() = default;
    void DisplayMainMenu();
```

```
};

static inline int TimeStringToMinutes(const std::string &raw) {
    std::string s = raw;
    while (!s.empty() && std::isspace(static_cast<unsigned
char>(s.front()))) s.erase(s.begin());
    while (!s.empty() && std::isspace(static_cast<unsigned
char>(s.back()))) s.pop_back();
    if (s.empty()) return -1;

    std::string upper = s;
    for (auto &c : upper) c =
static_cast<char>(std::toupper(static_cast<unsigned char>(c)));
    bool hasAM = (upper.find("AM") != std::string::npos);
    bool hasPM = (upper.find("PM") != std::string::npos);

    if (hasAM || hasPM) {
        std::string tmp;
        for (char c : s) if (!std::isalpha(static_cast<unsigned
char>(c))) tmp.push_back(c);
        s = tmp;
    }

    int hour = 0, minute = 0;
    size_t pos = s.find(':');
    if (pos == std::string::npos) pos = s.find('.');
    if (pos == std::string::npos) {
        try { hour = std::stoi(s); minute = 0; } catch (...) { return
-1; }
    } else {
        std::string h = s.substr(0, pos);
        std::string m = s.substr(pos+1);
        try { hour = std::stoi(h); minute = std::stoi(m); } catch (...)
{ return -1; }
    }
```

```
    if (hour < 0 || minute < 0 || minute >= 60) return -1;

    if (hasAM || hasPM) {
        if (hour == 12) {
            if (hasAM) hour = 0;
        } else {
            if (hasPM) hour = (hour % 12) + 12;
        }
    }

    if (hour < 0 || hour >= 24) return -1;
    return hour * 60 + minute;
}


static inline int GenerateUnique4DigitId(const std::vector<Patient>
&existing) {
    std::random_device rd;
    static std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1000, 9999);

    std::unordered_map<int, bool> used;
    used.reserve(existing.size() * 2 + 10);
    for (const auto &p : existing) used[p.id] = true;

    for (int attempt = 0; attempt < 10000; ++attempt) {
        int cand = dist(gen);
        if (used.find(cand) == used.end()) return cand;
    }

    for (int cand = 1000; cand <= 9999; ++cand) {
        if (used.find(cand) == used.end()) return cand;
    }
```

```
    return -1;
}

inline void PatientSystem::EditPatientRecord(int idToEdit) {
    auto it = std::find_if(patientList.begin(), patientList.end(),
                           [idToEdit](const Patient& p) { return p.id ==
idToEdit; });

    if (it != patientList.end()) {
        std::cout << "\n--- Editing Patient ID: " << it->id << " ---\n";
        std::cout << "Current Name: " << it->name << "\n";
        std::cout << "Current Age: " << it->age << "\n";

        std::cout << "Enter new name (leave blank to keep current): ";
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        std::string temp;
        std::getline(std::cin, temp);
        if (!temp.empty()) it->name = temp;

        std::cout << "Enter new age (enter 0 to keep current): ";
        int newAge = 0;
        if ((std::cin >> newAge)) {
            if (newAge > 0) it->age = newAge;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');

        std::cout << "Enter new contact number (leave blank to keep
current): ";
        std::string newContact;
        std::getline(std::cin, newContact);
        if (!newContact.empty()) it->contactNumber = newContact;
```

```
        std::cout << "Enter new gender (M/F/O) (leave blank to keep
current): ";
        std::string newGender;
        std::getline(std::cin, newGender);
        if (!newGender.empty()) it->gender = newGender;

        std::cout << "Enter new medical history summary (leave blank to
keep current): ";
        std::string newHistory;
        std::getline(std::cin, newHistory);
        if (!newHistory.empty()) it->history = newHistory;

        std::cout << "\nPatient record updated successfully!\n";
    } else {
        std::cout << "\nPatient ID " << idToEdit << " not found.\n";
    }
}

inline void PatientSystem::ScheduleAppointment() {
    std::cout << "\n--- Schedule New Appointment ---\n";
    if (patientList.empty()) {
        std::cout << "No patients available to schedule an
appointment.\n";
        return;
    }

    std::cout << "Available Patients:\n";
    for (const auto& p : patientList) {
        std::cout << "ID: " << p.id << ", Name: " << p.name << ", Age: "
<< p.age << "\n";
    }

    int selectedId = -1;
    std::cout << "Enter Patient ID for appointment (or enter 0 to select
the most recently added patient): ";
```

```
    std::string selStr;
    if (!(std::cin >> selStr)) {
        std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Invalid input.\n";
        return;
    }
    try { selectedId = std::stoi(selStr); } catch (...) { std::cout <<
"Invalid ID format.\n"; return; }
    if (selectedId == 0) {
        if (patientList.empty()) { std::cout << "No patients in the
system to select.\n"; return; }
        selectedId = patientList.back().id;
        std::cout << "Auto-selected Patient ID: " << selectedId << "\n";
    }

    auto it = std::find_if(patientList.begin(), patientList.end(),
                           [selectedId](const Patient& p) { return p.id
== selectedId; });
    if (it == patientList.end()) { std::cout << "Invalid Patient ID.
Returning to appointment menu.\n"; return; }

    Appointment newApp; newApp.patientId = selectedId;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    while (true) {
        std::cout << "Enter preferred appointment time (e.g., 10:00 AM)
or C to cancel: ";
        std::getline(std::cin, newApp.time);
        if (!newApp.time.empty() && (std::toupper(static_cast<unsigned
char>(newApp.time[0])) == 'C')) {
            std::cout << "Appointment scheduling cancelled.\n"; return;
        }
        int minutes = TimeStringToMinutes(newApp.time);
```

```
        if (minutes < 0) { std::cout << "Invalid time format. Please use
formats like 10:30 AM or 14:30." << "\n"; continue; }
        break;
    }

    // Fixed schedules per doctor (ranges are inclusive: [start, end])
    static const std::unordered_map<std::string,
std::vector<std::pair<int,int>>> doctorSchedules = {
        {"Dr. Owen",     {{9*60, 12*60-1}, {13*60, 17*60-1}}},
        {"Dr. Escalona", {{8*60, 12*60-1}, {13*60, 16*60-1}}},
        {"Dr. Crishen",  {{10*60, 14*60-1}}},
        {"Dr. Pizzaro",  {{9*60, 11*60-1}, {14*60, 18*60-1}}},
        {"Dr. Punay",    {{8*60, 12*60-1}, {12*60+30, 15*60-1}}}
    };

    auto isWorking = [&](const std::string &doc, int minute) -> bool {
        auto itSched = doctorSchedules.find(doc);
        if (itSched == doctorSchedules.end()) return false;
        for (const auto &r : itSched->second) if (minute >= r.first &&
minute <= r.second) return true;
        return false;
    };

    int requestedMinutes = TimeStringToMinutes(newApp.time);
    std::vector<std::string> availableDoctors;
    for (const auto &entry : doctorSchedules) {
        const std::string doc = entry.first;
        if (!isWorking(doc, requestedMinutes)) continue;
        bool occupied = false;
        for (const auto &app : appointmentQueue) {
            if (app.doctor == doc) {
                int appMin = TimeStringToMinutes(app.time);
                if (appMin >= 0 && appMin == requestedMinutes) {
occupied = true; break; }
            }
```

```
        }
        if (!occupied) availableDoctors.push_back(doc);
    }

    if (availableDoctors.empty()) { std::cout << "There are no doctors
available at that time. Returning to appointment menu.\n"; return; }

    std::cout << "\nDoctors available at " << newApp.time << ":\n";
    for (size_t i = 0; i < availableDoctors.size(); ++i) std::cout <<
(i+1) << ". " << availableDoctors[i] << "\n";

    int docChoice = -1;
    while (true) {
        std::cout << "Enter doctor number to choose, or 0 to auto-assign
the first available: ";
        if (!(std::cin >> docChoice)) { std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::cout << "Invalid input. Please enter a number.\n"; continue; }
        if (docChoice == 0) { newApp.doctor = availableDoctors[0];
break; }
        if (docChoice >= 1 && static_cast<size_t>(docChoice) <=
availableDoctors.size()) { newApp.doctor =
availableDoctors[docChoice-1]; break; }
        std::cout << "Invalid selection. Try again.\n";
    }

    std::cout << "\n--- Confirmation ---\n";
    std::cout << "Patient: " << it->name << " (ID: " << it->id << ")\n";
    std::cout << "Time: " << newApp.time << "\n";
    std::cout << "Doctor: " << newApp.doctor << "\n";

    char saveChoice;
    std::cout << "Save this appointment? (Y/N): ";
    std::cin >> saveChoice;
    if (std::tolower(static_cast<unsigned char>(saveChoice)) == 'y') {
```

```
        appointmentQueue.push_back(newApp);
        std::sort(appointmentQueue.begin(), appointmentQueue.end(),
[](const Appointment &a, const Appointment &b) {
            return TimeStringToMinutes(a.time) <
TimeStringToMinutes(b.time);
        });
        std::cout << "Appointment saved successfully!\n";
    } else { std::cout << "Appointment cancelled.\n"; return; }

    char addAnother;
    std::cout << "Do you want to schedule another appointment? (Y/N): ";
    std::cin >> addAnother;
    if (std::tolower(static_cast<unsigned char>(addAnother)) == 'y')
ScheduleAppointment();
}

inline void PatientSystem::AddNewPatientFunction() {
    std::cout << "\n\n--- Add New Patient ---\n";
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    while (true) {
        Patient newPatient;
        std::string line;

        std::cout << "Enter Patient Name: ";
        std::getline(std::cin, newPatient.name);

        while (true) {
            std::cout << "Enter Age: ";
            std::getline(std::cin, line);
            if (line.empty()) { std::cout << "Age cannot be empty.
Please enter a valid age.\n"; continue; }
            std::istringstream iss(line);
            if (iss >> newPatient.age && newPatient.age > 0) break;
            std::cout << "Invalid age input. Please enter a positive
number.\n";
```

```
        }

        std::cout << "Enter Contact Number: ";
        std::getline(std::cin, newPatient.contactNumber);

        std::cout << "Enter Gender (M/F/O): ";
        std::getline(std::cin, newPatient.gender);
        std::string hxChoice;
        while (true) { std::cout << "Does the patient have a medical
history to record? (Y/N): "; std::getline(std::cin, hxChoice); if
(!hxChoice.empty()) break; std::cout << "Please enter Y or N.\n"; }
        if (std::tolower(static_cast<unsigned char>(hxChoice[0])) ==
'y') {
            std::cout << "Enter Medical History Summary: ";
std::getline(std::cin, newPatient.history);
            if (newPatient.history.empty()) newPatient.history =
"(history provided but left blank)";
        } else {
            newPatient.history = "No known medical history";
        }

        if (!newPatient.name.empty() && newPatient.age > 0) {
            int generatedId = GenerateUnique4DigitId(patientList);
            if (generatedId < 0) { std::cout << "Failed to generate a
unique patient ID. Please try again later.\n"; continue; }
            newPatient.id = generatedId;
            patientList.push_back(newPatient);
            std::cout << "\nPatient added to the list successfully (ID:
" << newPatient.id << ").\n";
        } else { std::cout << "Patient name or age is invalid. Please
try again.\n"; continue; }

        std::cout << "Do you want to add another patient? (Y/N): ";
        std::getline(std::cin, line);
```

```
        if (line.empty() || std::tolower(static_cast<unsigned
char>(line[0])) != 'y') break;
    }
}

inline void PatientSystem::PatientListFunction() {
    while (true) {
        std::cout << "\n\n--- Patient List ---\n";
        if (patientList.empty()) { std::cout << "No patients
recorded.\n"; }
        else {
            for (const auto& p : patientList) {
                std::cout << "ID: " << p.id << ", Name: " << p.name
                          << ", Age: " << p.age
                          << ", Contact: " << p.contactNumber
                          << ", Gender: " << p.gender
                          << ", History: " << p.history << "\n";
            }
        }

        std::cout << "\nOptions:\n";
        std::cout << "1. Edit patient record\n";
        std::cout << "2. Delete a patient record\n";
        std::cout << "3. Return to main menu\n";
        std::cout << "Select an option: ";

        int choice;
        if (!(std::cin >> choice)) { std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::cout << "Invalid input. Please enter a number.\n"; continue; }

        if (choice == 1) { int idToEdit; std::cout << "Enter Patient ID
to Edit: "; std::cin >> idToEdit; EditPatientRecord(idToEdit); }
        else if (choice == 2) {
```

```
            int idToDelete; std::cout << "Enter Patient ID to Delete: ";
std::cin >> idToDelete;
            size_t initialSize = patientList.size();
            patientList.erase(std::remove_if(patientList.begin(),
patientList.end(), [idToDelete](const Patient& p){ return p.id ==
idToDelete; }), patientList.end());
            if (patientList.size() < initialSize) {

appointmentQueue.erase(std::remove_if(appointmentQueue.begin(),
appointmentQueue.end(), [idToDelete](const Appointment& app){ return
app.patientId == idToDelete; }), appointmentQueue.end());
                std::cout << "Patient ID " << idToDelete << " deleted
successfully." << "\n";
            } else { std::cout << "Patient ID " << idToDelete << " not
found.\n"; }
        } else if (choice == 3) { break; }
        else { std::cout << "Invalid choice. Please try again.\n"; }
    }
}

inline void PatientSystem::PatientAppointmentFunction() {
    while (true) {
        std::cout << "\n\n--- Patient Appointment Menu ---\n";
        std::cout << "1. Display schedule\n";
        std::cout << "2. Appoint a schedule for patient\n";
        std::cout << "3. Return to main menu\n";
        std::cout << "Select an option: ";

        int choice;
        if (!(std::cin >> choice)) { std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::cout << "Invalid input. Please enter a number.\n"; continue; }

        if (choice == 1) {
            std::cout << "\n--- List of Scheduled Patients ---\n";
```

```
            if (appointmentQueue.empty()) std::cout << "No appointments
scheduled.\n";
            else {
                for (const auto& app : appointmentQueue) {
                    auto it = std::find_if(patientList.begin(),
patientList.end(), [app](const Patient& p){ return p.id ==
app.patientId; });
                    std::string patientName = (it != patientList.end())
? it->name : "Unknown";
                    std::cout << "Patient: " << patientName << ", Time:
" << app.time << ", Doctor: " << app.doctor << "\n";
                }
            }
            char scheduleAgain; std::cout << "Do you want to schedule a
patient now? (Y/N): "; std::cin >> scheduleAgain;
            if (std::tolower(static_cast<unsigned char>(scheduleAgain))
== 'y') ScheduleAppointment(); else break;

        } else if (choice == 2) ScheduleAppointment();
        else if (choice == 3) break;
        else std::cout << "Invalid choice. Please try again.\n";
    }
}

inline void PatientSystem::DisplayMainMenu() {
    int choice;
    do {
        std::cout << "\n\n==========================\n";
        std::cout << "  Patient Management System\n";
        std::cout << "==========================\n";
        std::cout << "1. Add New Patient Info\n";
        std::cout << "2. Patient List (Edit/Delete)\n";
        std::cout << "3. Patient Appointment Schedule\n";
        std::cout << "4. Exit\n";
        std::cout << "Select an option: ";
```

```
        if (!(std::cin >> choice)) { std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::cout << "Invalid input. Please enter a number.\n"; continue; }

        switch (choice) {
            case 1: AddNewPatientFunction(); break;
            case 2: PatientListFunction(); break;
            case 3: PatientAppointmentFunction(); break;
            case 4: std::cout << "Exiting system. Goodbye!\n"; break;
            default: std::cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 4);
}

#endif
```

Figure 10: patientSystem.h

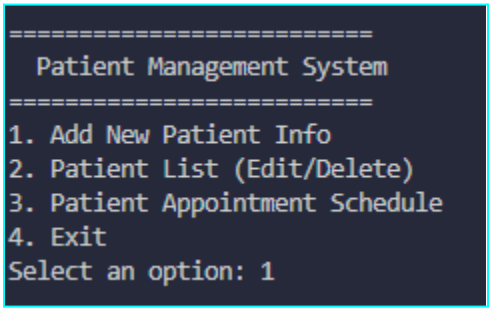

Figure 11: code output 1

```
--- Add New Patient ---
Enter Patient Name: Francis Nikko I. Andoy
Enter Age: 20
Enter Contact Number: 096123123
Enter Gender (M/F/O): M
Does the patient have a medical history to record? (Y/N): N

Patient added to the list successfully (ID: 5186).
Do you want to add another patient? (Y/N): Y
Enter Patient Name: Crishen Pulgado
Enter Age: 21
Enter Contact Number: 4124231241
Enter Gender (M/F/O): F
Does the patient have a medical history to record? (Y/N): N

Patient added to the list successfully (ID: 6415).
Do you want to add another patient? (Y/N): N
```

Figure 12: 2nd code output

```
========================
  Patient Management System
========================
1. Add New Patient Info
2. Patient List (Edit/Delete)
3. Patient Appointment Schedule
4. Exit
Select an option: 2


--- Patient List ---
ID: 5186, Name: Francis Nikko I. Andoy, Age: 20, Contact: 096123123, Gender: M, History: No known medical history
ID: 6415, Name: Crishen Pulgado, Age: 21, Contact: 4124231241, Gender: F, History: No known medical history

Options:
1. Edit patient record
2. Delete a patient record
3. Return to main menu
Select an option: 1
Enter Patient ID to Edit: 5186
```

Figure 13: 3rd code output

```
--- Editing Patient ID: 5186 ---
Current Name: Francis Nikko I. Andoy
Current Age: 20
Enter new name (leave blank to keep current):
Enter new age (enter 0 to keep current):
0
Enter new contact number (leave blank to keep current):
Enter new gender (M/F/O) (leave blank to keep current):
Enter new medical history summary (leave blank to keep current):

Patient record updated successfully!
```

Figure 14: 4th code output

```
--- Patient List ---
ID: 5186, Name: Francis Nikko I. Andoy, Age: 20, Contact: 096123123, Gender: M, History: No known medical history
ID: 6415, Name: Crishen Pulgado, Age: 21, Contact: 4124231241, Gender: F, History: No known medical history

Options:
1. Edit patient record
2. Delete a patient record
3. Return to main menu
Select an option: 2
Enter Patient ID to Delete: 5186
Patient ID 5186 deleted successfully.
```

Figure 15: 5th code output

```
--- Patient List ---
ID: 6415, Name: Crishen Pulgado, Age: 21, Contact: 4124231241, Gender: F, History: No known medical history

Options:
1. Edit patient record
2. Delete a patient record
3. Return to main menu
Select an option: 3


========================
  Patient Management System
========================
1. Add New Patient Info
2. Patient List (Edit/Delete)
3. Patient Appointment Schedule
4. Exit
Select an option: 3
```

Figure 16: 6th code output

```
--- Patient Appointment Menu ---
1. Display schedule
2. Appoint a schedule for patient
3. Return to main menu
Select an option: 1

--- List of Scheduled Patients ---
No appointments scheduled.
Do you want to schedule a patient now? (Y/N): y

--- Schedule New Appointment ---
Available Patients:
ID: 6415, Name: Crishen Pulgado, Age: 21
Enter Patient ID for appointment (or enter 0 to select the most recently added patient): 6415
Enter preferred appointment time (e.g., 10:00 AM) or C to cancel: 7:00
There are no doctors available at that time. Returning to appointment menu.
```

Figure 17: 7th code output

```
--- Patient Appointment Menu ---
1. Display schedule
2. Appoint a schedule for patient
3. Return to main menu
Select an option: 2

--- Schedule New Appointment ---
Available Patients:
ID: 6415, Name: Crishen Pulgado, Age: 21
Enter Patient ID for appointment (or enter 0 to select the most recently added patient): 6415
Enter preferred appointment time (e.g., 10:00 AM) or C to cancel: 10:00

Doctors available at 10:00:
1. Dr. Punay
2. Dr. Pizzaro
3. Dr. Crishen
4. Dr. Escalona
5. Dr. Owen
Enter doctor number to choose, or 0 to auto-assign the first available: 1
```

Figure 18: 8th code output

```
--- Confirmation ---
Patient: Crishen Pulgado (ID: 6415)
Time: 10:00
Doctor: Dr. Punay
Save this appointment? (Y/N): y
Appointment saved successfully!
```

Figure 19: 9th code output

The figures above show the execution of the appointment system. It begins with the patient.h header file that contains the Patient and Appointment class with their corresponding class members: the ID, name, age, contactNumber, gender, and history for the patient while patientID, time, and doctor for the appointments. In the PatientSystem header, the patient.h class and the corresponding libraries used are initiated to simulate the patient system. The main driver simply creates a PatientSystem object to initialize the simulation using the DisplayMainMenu() member function.

CONCLUSION

In conclusion, the created dental appointment management system was able to develop a system for storing and managing patient information. It also includes a scheduling system that allows staff to manage patient appointments. Additionally, the system stores patient information and medical histories, allowing proper documentation and tracking. With proper documentation readily available, staff is able to accurately search for patient records and appointment details using the unique IDs generated by the program for each patient. Lastly, the system uses a simple interface to allow staff to navigate patient records and appointment schedules.

REFERENCES

Azzali, F., Abas, A., & Yahya, N. M. H. N. (2024). Enhancing dental service efficiency: Development and evaluation of a smart dental appointment system. *SODH SARITA*, 1–5. https://doi.org/10.1109/netapps63333.2024.10823486

Gomez, O., Juan, A., Tsvetanov, G., Muñoz, R., & Torres, R. (2023). Simulation the customer service capacity in a dental clinic. *International Multidisciplinary Modeling & Simulation Multiconference*. https://doi.org/10.46354/i3m.2023.mas.009

Ho, S., Chew, E., & Tan, C. (2024). Streamlining dental clinic management for effective digitisation productivity and usability. *Journal of Informatics and Web Engineering*, *3*(2), 70–85. https://doi.org/10.33093/jiwe.2023.3.2.5

Sihombing, D. (2024). Optimizing the Development of Online Appointment System for Dental Clinics through Agile Methodology Implementation. *Jurnal Ekonomi*, *13*(01). https://doi.org/10.54209/ekonomi.v13i01